Java 8

Practice Book

Document Revision History

Date	Revision No.	Author	Summary of Changes
September 2020	1.0	Neelima Padmawar	Practice Book

Table of Contents

STRINGS

1. Validation

Case Study Description :

Your task here is to implement a **Java** code based on the following specifications. Note that your code should match the specifications in a precise manner. Consider **default visibility** of classes, data fields and methods unless mentioned otherwise.

Specifications

```
class definitions:
   class TransactionParty:
      Variables:
         String seller
         String buyer
      Implement a Constructor using the class variables.
   class Receipt:
      Variables:
         TransactionParty transactionParty
         String productsQR
      Implement a Constructor using the class variables.
   class GenerateReceipt:
      method definitions:
         verifyParty(Receipt r): Use regular expression to verify if
the two names in the TransactionParty class is valid or not and return
number of verified names.
            Return type: int
            Visibility: public

         calcGST(Receipt r): extract the rate and quantity of the
products from input string and calculate GST @12%
            Return type: String
            Visibility: public
```

Tasks

- Implement the two classes **Receipt** and **TransactionParty** class, according to the Class Descriptions below.
- Implement two methods (**verifyParty** and **calcGST**) in the **GenerateReceipt** class, according to the Method Definitions below:

Class Descriptions

class TransactionParty

Implement a parameterized constructor to initialize these instance variables.

1. **seller** signifies the name of the seller of the products
2. **buyer** signifies the name of the buyer of the products

class Receipt

Implement a parameterized constructor to initialize these instance variables.

1. TransactionParty transactionParty

2. String productsQR

- a string of the format: "<Rate>,<Quantity>@<Rate>,<Quantity>@<Rate>,<Quantity>"
- e.g: "250,10@100,3@50,7"
- only 3 products' Rate and Quantity present in the string

To access a variable in TransactionParty class through Receipt object we use:

```
<Receipt(obj)>.<Receipt(variable)>.<TransactionParty(variable)>
```

e.g: To access "seller" name from the Receipt object r we use:

```
r.transactionParty.seller;
```

Method Descriptions

1. verifyParty(Receipt r):

In this method, you have to use regex to check if the names - buyer and seller of TransactionParty object available in Receipt r, are valid or not. Validation is based on:

- Names should contain only alphabets (uppercase/lowercase), a single whitespace or the symbols single quote and hyphen (' OR -). e.g: Daniel D'Cruz and Giselle Dawn-Wright are valid names.

Return:

- 2 if the both names are valid
- 1 if only one names is valid
- 0 if both names are invalid.

2. calcGST(Receipt r):

In this method, you have to use the productsQR variable of the Receipt r object to extract the Quantity and Rate of 3 products.

- The variable productsQR will have a string value of the format <Rate>,<Quantity>@<Rate>,<Quantity>@<Rate>,<Quantity>

- **Rate** is the price rate of the product
- **Quantity** is the number of units bought
- **e.g**: **productsQR** = "**250,10@100,3@50,7**" has the **Rate** as 250, 100 and 50 while **Quantity** as 10, 3 and 7 respectively.

Use the **GST_Rate** as **12%.**

Calculate the value of GST using the formula:

```
GST = (Rate1 * Quantity1 + Rate2 * Quantity2 + Rate3 * Quantity3) *
GST_Rate
```

The data type of GST should be **int** only. (Do NOT round-off the result)

Return the value of **GST** as a string value, using the **toString()** method.

2. Employee Information

Case Study Description :

Your task here is to implement a **JAVA** code based on the following specifications. Consider **default visibility** of classes, data fields and methods unless mentioned otherwise.

String manipulation is a tough task and your company wants you to do some string operations and manipulation.

Suppose you are given a string (example **Rakesh RaJ@1PC16CS046-SDE#8):**

- The part before **'@'** represents the name of the Employee.
- The part before **'-'** and after **'@'** represents the ssn of the Employee.
- The part before **'#'** and after **'-'** represents the dept of the Employee.
- The part after **'#'** represents the salary of the Employee.

Specifications:

```
class definitions:
  class Employee:
      data fields:
        name: String variable
        ssn: String Variable
        dept: String variable
        salary : int variable
      Constructor to initialize the class variables.
        public Employee(name,ssn,dept,salary)

  class EmployeeImplementation:
```

```
      method definitons:
        getEmployeeInfo(String str): To extract the name,ssn,dept and
salary from the String str and return an Employee object initialized
with extracted info.
            return type: Employee object
            visibility: public

        getEmployeeDept(Employee e): Use the ssn of the Employee e and
return the dept he/she is in. If last three digit of Employee ssn is
between 001-060 return "L1", if ssn is between 061-120 return "L2" ,
if ssn is between 121-180 return "L3" and if ssn greater than 180
return "L4".
            return type: String
            visibility: public
```

Task:

- Implement the **Employee** class according to the specification given.
- Implement the **EmployeeImplementation** class where you have to implement the following two methods according to the specifications given:

1. **Employee getEmployeeInfo(String str)**
2. **String getEmployeeDept(Employee s)**

Method Descriptions:

1. **Employee getEmployeeInfo(String str)**:

- takes a String parameter (e.g. **Amit Rai@1PC16CS046-ALU#8**).
- Extract the information from the String str.
- Populate an **Employee** object with the extracted information.
- Return the **Employee** object.

2. **String getEmployeeDept(Employee e)**:

- takes a **Employee** object as the parameter.
- Use the last three digits of **ssn** of the String str and return the **dept** he/she is in.
- If last three digit of **Employee's ssn** is between 001-060 return "**L1**", if **ssn** is between 061-120 return "**L2**" , if ssn is between 121-180 return "**L3**" and if **ssn** is greater than 180 return "**L4**".

COLLECTIONS

1. TV Show

Case Study Description :

Your task here is to implement a Java code based on the following specifications. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
class definitions:
 class Source:
     method definitons:
       printIndex(ArrayList<String> list, int ind): Method to return
the String at index ind
           return type: String
           visibility: public
           return: String at index ind
       addAfter(ArrayList<String> a, String m, String n): Method to
add element n after element m
           return type: ArrayList<String>
           visibility: public
           return: List containing element n after m
       pickIndexAndAppend(ArrayList<String> p, int ind): Method to
pick a string at index ind and append to the end of the list
           return type: ArrayList<String>
           visibility: public
```

Task:

Your task is to create a **Source** class based on the above specifications and implement the following:

- **printIndex(ArrayList<String> list, int ind):** Method to return the element present at index **ind** from ArrayList **list**
- **addAfter(ArrayList<String> a, String m, String n):** Method to add string **n** after the string element **m** in ArrayList **a**.
- **pickIndexAndAppend(ArrayList<String> p, int ind):** Method to pick the String at index **ind** from ArrayList **p** and append it to the end of the list.

Sample Output:

```
Friends
[Breaking Bad, Young Sheldon, Friends, Sherlock, Stranger Things]
[Breaking Bad, Young Sheldon, Sherlock, Stranger Things, Friends]
```

2. Set Operation

John has recently learned about **HashSet** in Java. He wants to implement the basic operations like ***subtract, union, intersect*** using **HashSet**. Help John in implementing the basic functionalities.

Your task here is to implement a Java code based on the following specifications. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
class definitions:
 class Source:
  method definitons:
  subtract(Set<Integer> a, Set<Integer> b): method to subtract hashset
a and b in which you have to remove all the elements of a which are in
b
       return type: Set<Integer>
       visibilty: public

   union(Set<Integer> a, Set<Integer> b): method to perform union on
HashSet's a and b
       return type: Set<Integer>
       visibilty: public

   intersect(Set<Integer> a, Set<Integer> b): method to perform
intersection on HashSet's a and b
       return type: Set<Integer>
       visibilty: public
```

Task:

Your task is to create a **Source** and implement the following:

- **Set<Integer> subtract(Set<Integer> a, Set<Integer> b)** method to return the **subtracted** element, subtract all the elements of a which are in b and return the set
- **Set<Integer> union(Set<Integer> a, Set<Integer> b)** method to return the union of two sets
- **Set<Integer> intersect(Set<Integer> a, Set<Integer> b)** method to return the intersection of two sets.

Sample Input

```
HashSet<Integer> set1 = new HashSet();
       set1.add(5);
       set1.add(6);
       set1.add(7);
```

```
        set1.add(8);
HashSet<Integer> set2 = new HashSet();
        set2.add(9);
        set2.add(3);
        set2.add(7);
```

Sample Output

```
[5, 6, 8]
[3, 5, 6, 7, 9]
[7]
```

3. String Position

Given a string k consisting of lowercase letters, these letters form consecutive groups of the same character.

For example, a string like K = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z" and "yy".

A group is called *large* if it has 3 or more characters. Your task is to return the starting and ending positions of every large group.

Implement a Java code based on the following specifications. Note that your code should match the specifications in a precise manner. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
 class Source:
  method definitons:
    printPositions(String K): Method to return the starting and ending
positions of every large group.
      return type: List<List<Integer>>
      visibilty: public
    addAfter(ArrayList<String> a, String m, String n): Method to add
string n at the position next to the position of string element m in
the list.
      return type: ArrayList<String>
      visibility: public
```

Task:

Create a Source class and implement below given method:

- **printPositions(String K):** Method to **return** the **starting** and **ending** positions of every large group.
- **addAfter(ArrayList<String> a, String m, String n):** Method to add string n at the position next to the position of string element m in the list.(Hint: Add n after every occurrence of m in a)

Algorithm:

printPositions(String K):

- Maintain pointers i, j with i <= j. The i pointer will represent the start of the current group, and we will increment j forward until it reaches the end of the group.
- We know that we have reached the end of the group when j is at the end of the string, or S[j] != S[j+1]. At this point, we have some group [i, j]; and after, we will update i = j+1, the start of the next group.

Sample Input:

```
mousssssseeee
ArrayList<String> list = new ArrayList<>();
list.add("ad");
list.add("cc");
list.add("df");
list.add("ez");
m = cc, n = kc;
```

Sample Output:

```
[[3, 8], [9, 12]]
[ad, cc, kc, df, ez]
```

Explanation:

- In the above example the string **"kc"** denoted by string **n** is placed at position next to position of string **"cc"** denoted by string **m.**

4. Longest SubString

You're given a **string**, your task is to find the length of the **longest substring** without repeating characters.

Your task here is to implement a **Java** code based on the following specifications. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
class definitions:
 class Source:
  method definitons:
       lengthOfLongestSubstring(String s, Set<Character> set):
           return type: int
```

```
            visibilty: public
```

Task:

Class Source

Implement the below method for this class:

- int lengthOfLongestSubstring(String s, Set<Character> set) : Method to find the length of the longest substring without repeating characters.(Use HashSet to store the characters)

Algorithm:

Using sliding window technique

- Use HashSet to store the characters in current window [i,j)[i, j)[i,j) (j=ij = ij=i initially).
- Slide the index jjj to the right. If it is not in the HashSet, slide jjj further. Doing so until s[j] is already in the HashSet.
- At this point, You will find the maximum size of substrings without duplicate characters start with index iii. If you do this for all iii, you will get your answer.

Sample Input

```
abcabcbb
```

Sample Output

```
3(answer is "abc", with the length of 3)
```

5. Students Information

Your task here is to implement a Java code based on the following specifications. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
class definitions:
 class Source:
   method definitons:
     changeOccurrence(ArrayList<String> a,String m,String n): Method to change all occurrences of m to
n in the arrayList
         return type: ArrayList<String>
         visibility: public

     listIndex(ArrayList<String> list): Method to return the element present in list at index 0
```

```
    return type: String
    visibility: public

listAfter(ArrayList<String> a, String m, String n): Method to add element n after element m
    return type: ArrayList<String>
    visibility: public
```

Task:

Your task is to create a Source class based on the above specifications and implement the following:

- changeOccurrence(ArrayList<String> a, String m, String n): Method to change all occurences of m to n in the ArrayList
- listIndex(ArrayList<String> list): Method to return the element present in list at index 0 (You can use get() method to get the element present in the list at a given specific index.)
- listAfter(ArrayList<String> a, String m, String n): Method to add string n after the string element m and return the list containing all the elements with n after m

Sample Output:

```
[A, B, S, D]
A
[A, B, C, D, E]
```

6. Anagrams

Description:

Given a string s and a non-empty string p, your task here is to find all the start indices of p's anagrams in s.

Implement a Java code based on the following specifications. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
class definitions:class Source:
       method definition:
              findAnagrams(String s, String p): method to find all the
start indices of p's anagrams in s.
                     return type: List<Integer>
                     visibility: public
```

Task

Given a Source class, Implement the below given method

- **findAnagrams(String s, String p):** Method to find all the start indices of **p**'s anagrams in **s.**

  Algorithm:

- Initialize a List
- Count number of different characters in p
- Initialize start and end index of the substring
- Initialize the counter
- Add character to the right of substring and modify the counter if necessary
- If end > p.length(), every time we add a character, we also need to delete a character from the left of substring to make the length of substring equal to p.length(), then we modify the counter if necessary
- If count is equal to number of different characters,then the substring is an anagram of p, add the index to the list

  Input

```
s: "cbaebabacd" p: "abc"
```

Output

```
[0, 6]
```

Explanation

The substring with start index = 0 is "cba", which is an anagram of "abc".

The substring with start index = 6 is "bac", which is an anagram of "abc".

7. Encryption

Description

For telecommunication a method known as **Morse Code** is used to encode text characters as standardized sequences of two different signal durations, called dots and dashes or dits and dahs such as : "a" maps to ".-", "b" maps to "-..."，"c" maps to "-.-.", and so on.

The full table for the 26 letters of the English alphabet is given below for your convenience:

```
[".-","-...","-.-.","-..",".","..-.","--.","....","..",".---","-.-
",".-..","--","-.","---",".--.","--.-",".-.","...","-","..-","...-
",".--","-..-","-.--","--.."]
```

Your task here is to implement a **Java** code based on the following specifications. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
class definitions:
 class Source:
 method definition:
  uniqueMorseRepresentations(String[] words): return the number of
unique morse transformations
      return type: int
      visibility: public
```

Task:

Given a list of words, each word can be written as a concatenation of the Morse code of each letter.

For example, "cba" can be written as "-.-..--...", (which is the concatenation "-.-." + "-..." + ".-"). We'll call such a concatenation, the transformation of a word.

Based on the above specifications , implement the below given method:

- uniqueMorseRepresentations(String[] words): Method that will return the number of unique morse transformations.

Important:

- The length of words does not exceed 100.
- Each words[i] will have length in range [1, 12].
- words[i] will only consist of lowercase letters.

Approach:

- Accept an array of strings
- Convert each string in the array into morse code
- create hashset/ hashmap to store these morsecodes with their strings
- return the unique morse code

Sample Input:

```
String[] MORSE = new String[]{"gin", "zen", "gig", "msg"}
```

Sample Output:

```
2
```

Explanation:

```
Explanation:
The transformation of each word is:
"gin" -> "--...-."
"zen" -> "--...-."
"gig" -> "--...--."
```

```
"msg" -> "--...--."

There are 2 different transformations, "--...-." and "--...--.".
```

EXCEPTIONS

1. find Age

Description :

Your task here is to implement a Java code based on the following specificationsConsider **default visibility** of class unless mentioned otherwise.

Specifications:

```
class definitions:
    class Age:
        class variables:
            String drink
            String vote
            String movie


    class ExceptionCheck:
        class Methods:
            drinkingCheck(Age a, int age): This method has an
object  of Age class and age(integer) as parameter.
                    Visibility: public
                    return type: String

            votingCheck(Age a, int age):  This method has an object of
Age class and age(integer) as the parameter.
                    Visibility: public
                    return type: String

            movieCheck(Age a, int age):  This method has an object of
Age class and age(integer) as the parameter.
                    Visibility: public
                    return type: String

    class IllegalAgeException extends Exception:    //User-Defined
Exception class extends Exception class
        IllegalAgeException(String s):
            visibility : public
```

Tasks:

- Implement the **Age** class with the variables **drink, vote and movie** all of type String.
- Implement **ExceptionCheck** class with three **methods**:

1. **drinkingCheck (Age a, int age)**
2. **votingCheck (Age a, int age)**
3. **movieCheck (Age a, int age)**

- Implement user-defined Exception class **IllegalAgeException** which extends **Exception** class. The class will have a **constructor** which will be used to initialise the class with the message.

  Method Descriptions:

  1. **drinkingCheck(Age a, int age)**:

- You have to implement **try-catch** block to check if **age < 21**
- If the age is less than 21 then assign **Age class (a.drink)** drink variable as "**illegal**" and throw an user-defined exception **IllegalAgeException("Illegal drinking age")** .
- If the age is greater than or equal to 21 then assign drink variable as "**legal**".
- Return default message if exception is thrown else return **a.drink**.

  2. **votingCheck(Age a, int age):**

- You have to implement **try-catch** block to check if **age < 18**
- If age is less than 18, then assign **Age class (a.vote)** vote variable as "**illegal**" and throw an user-defined exception **IllegalAgeException("Illegal voting age")**.
- If the age is greater than or equal to 18, then assign vote variable as "**legal**".
- Return default message if an exception is thrown else return **a.vote**.

  3. **movieCheck(Age a, int age):**

- You have to implement **try-catch** block to check if **age < 14**
- If age is less than 14, then assign **Age class (a.movie)** movie variable as "**illegal**" and throw an user-defined exception **IllegalAgeException("Illegal movie-watching age")** .
- If the age is greater than or equal 14, assign movie variable as "**legal**".
- Return default message if an exception is thrown else return **a.movie**.

2. File Check

Description

Your task here is to implement a Java code based on the following specifications. Note that your code should match the specifications in a precise manner. Consider **default visibility** of class unless mentioned otherwise.

```
class definitions:
       class ExceptionCheck:
```

```
numberCheck(String str): Check each character of String
str, if its a number between 0-9 add it to an ArrayList<String> else
catch exception NumberFormatException and add the default exception
message to the ArrayList. Return the ArrayList<String> object. Use
try-catch.
                    Visibility:public
                    Return type: ArrayList<String>


fileCheck(String filename): Given a string filename,
check if the file exists in the current directory if found return
"File Found" else catch FileNotFoundException and return the default
exception method. Use try-catch.
                    Visibility: public
                    Return type: String
```

TASK:

- Implement ExceptionCheck class with the two methods explained below.
- The methods to be Implemented are:

1. numberCheck(String str)
2. fileCheck(String filename)

Methods to be Implemented:

1. numberCheck(String str):

- Use try-catch to implement the method.
- Check each character of String str, if its a number between 0-9.
- If its a number add it to an ArrayList<String> else catch exception NumberFormatException and add the default exception message to the ArrayList. Return the ArrayList<String> object. Use try-catch.
- e.g: str = "10ASD"
- ArrayList will return: [1, 0, For input string: "A", For input string: "S", For input string: "D"]

2. fileCheck(String filename):

- Use try-catch to implement the method.
- Given a string filename, check if the file exists in the current directory.
- If found return "File Found" else catch FileNotFoundException and return the default exception method. Use try-catch.
- e.g: filename = "abc.txt"
- if abc.txt is present it will return "File Found". If the file is not present in the current directory return string will be default message thrown from the exception: FileNotFoundException.

3. Email Operation

Description

Your task here is to implement a Java code based on the following specifications. Consider default visibility of classes, data fields and methods unless mentioned otherwise.

Specifications:

```
class definitions:
   class Header:
              Variables:
                     String from
                     String to
              Implement a parameterized constructor to initialize all
the instance variables.

   class Email:
              Variables:
                     Header header
                     String body
                     String greetings
              Implement a parameterized constructor to initialize all
the instance variables.
  class EmailOperations:
              Methods:
                     emailVerify(Email e): Use regular expression to
verify if the two email-ids in the Header class is valid or
not.[Return type explained in Task part].
                            Return type:int
                            Visibility: public

                     bodyEncryption(Email e): Use Ceasar cipher(Shift-
3) to encrypt the body of the email.[To know more refer the Task
part]
                            Return type:String
                            Visibility: public

                     greetingMessage(Email e): In this method you have
to return a greeting messgae. The greet part should be taken from
greetings variable and signature(name) should be taken from Header's
'from' email address.[To know more refer the Task part]
                            Return type:String
                            Visibility: public
```

Class Variables:

- class Header: It contains two email id 'from' and 'to'. 'from' signifies the sender's email address and 'to' signifies receiver's email address.
- class Email: This class contains three parts: first Header header which has two email address from and to,the second body which contains the message to send and third greetings which contains greetings such as "Regards", "Thank you", etc.

To access a variable in Header class through Email object we use:

```
<Email(obj)>.<Email(variable)>.<Header(variable)>
```

Example to access "from" address from the Email object e we use : e.header.from;

Tasks:

- Implement the two classes Email and Header class according to the specifications.
- Implement the three methods in the EmailOperations class:

1. emailVerify (Email e)
2. bodyEncryption (Email e)
3. greetingMessage (Email e)

Method Description:

1. emailVerify(Email e):

- In this method you have to use regex to check if the email-address to and from in Header class is valid or not. Validation is based on:
- Email address should start with alphabets(capital/small) or _(underscore).
- Email address should have only one @ followed by alphabets.
- Email address should end with .(dot) followed by alphabets.
- e.g: amit@doselect.com, _ami@doselect.in are valid addresses, but 1ami@dos.com, amit@doselect are invalid addresses.
- Return 2 if the both email addresses are valid return 1 if one is valid, and 0 if both are invalid.

2. bodyEncryption(Email e):

- In this method, you have to use Caesar cipher(shift of 3) to encrypt the body part of the Email return the encrypted string.
- Caesar shift, is one of the simplest and most widely known encryption techniques. It is a type of substitution cipher in which each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet. Here the number of shift is 3.
- e.g: str = "Hi There Hows you", after encryption becomes "Kl Wkhuh Krzv brx". H get converted to K that is a shift of 3 alphabets ahead.
- Letters which are capital should be capital and small should be small in Encrypted message. Take care of the spaces.

3. greetingMessage(Email e):

- In this method, you have to return a concatenated string which contains the greetings variable from Email class and Name of the person who is sending the mail(from variable in the Header class).
- The name part should not contain anything which is after @ in the email id.
- e.g: if greetings = "Regards" and from = "Amit@doselect.com" then you have to return the message "Regards Amit"

4. Custom Exceptions

Problem Statement

Marcus is an avid programmer by choice and a transporter by profession. To create and automate a database that would list his current vehicles, he decides to program a system for it. You, being his work partner, have been allotted the job of developing the back-end of this system. The exact task description is as follows:

Task

Create an exception named *TypeException.*

```
Overrides the toString() method:
   It will return a String "Vehicle Type Not Set".
```

You need to create a class named ***Vehicles***:

```
Data members (All Strings):
   type, model_no, model_name, owner_name, owner_details

Constructor that will take following parameters as input and will set them:
   model_no, model_name, owner_name, owner_details

And a default constructor too.

Following methods:
   get_type: returns the value of type
   retrieve: returns a string. Here, will return "null" as String.
```

Another class named ***Car*** which will inherit ***Vehicles*** class:

```
Constructor that will take the parameters as input and will set them
   Will call super Constructor with the input parameters.

And a default constructor too.

Following methods:
```

```
get_type: returns the value of type
set_type: setter method with void return type
retrieve:
  Overrides the Vehicles retrieve method.
  if type is null, it throws a TypeException.
  else it returns the string concatenating model_no, model_name, owner_details and owner_name
```